

Module 2.2: Distributed Data Models

Contents

NoSQL Databases	Error! Bookmark not defined.
Need for NoSQL Databases	Error! Bookmark not defined.
Characteristics of NoSQL Databases	Error! Bookmark not defined.
Advantages of NoSQL Databases	Error! Bookmark not defined.
Aggregate Data Models	Error! Bookmark not defined.
Importance of Aggregate Data Models	Error! Bookmark not defined.
Benefits of Aggregate-Oriented Design	Error! Bookmark not defined.
Aggregates	Error! Bookmark not defined.
Aggregate Boundaries	Error! Bookmark not defined.
Uses of Aggregates	Error! Bookmark not defined.
Advantages of Aggregates	Error! Bookmark not defined.
Key-Value Data Model	Error! Bookmark not defined.
Document Data Model	Error! Bookmark not defined.
Relationships in NoSQL Databases	Error! Bookmark not defined.
Graph Databases	Error! Bookmark not defined.
Nodes	Error! Bookmark not defined.
Edges	Error! Bookmark not defined.
Properties	Error! Bookmark not defined.
Schemeless Databases	Error! Bookmark not defined.
Materialized Views	Error! Bookmark not defined.
Working of Materialized Views	Error! Bookmark not defined.

Distributed Data Models

A Distributed Data Model is a database model in which data is stored across multiple physical locations, servers, or sites connected through a network. Although the data is distributed, it appears to users as a single unified database. Distributed data models improve performance, availability, reliability, and scalability by sharing data among different locations.

Need for Data Distribution

Data distribution is necessary in modern database systems for several reasons:

- **Improved Performance:** Data can be accessed from the nearest location, reducing response time.
- **Scalability:** Additional servers can be added to handle increasing data and user demands.
- **High Availability:** Data remains accessible even if one site or server fails.
- **Reliability:** Multiple copies of data reduce the risk of data loss.
- **Load Balancing:** Workload can be distributed across multiple servers.
- **Geographical Distribution:** Supports organizations operating in multiple locations.
- **Reduced Network Traffic:** Local access to data minimizes network congestion.

Types of Distribution Models

A. Partitioned (Fragmented) Distribution

Model: In this model, the database is divided into smaller fragments, and each fragment is stored at a different site. Each site manages only its assigned portion of the data.

Advantages:

- Reduces storage requirements at individual sites.
- Improves query performance for local users.
- Supports efficient data management.

Example: Customer records are divided among branches based on geographical regions.

B. Replicated Distribution Model: In this model, copies of the same data are stored at multiple sites. Every site contains identical or partially identical data. **Advantages:**

- Increases data availability.
- Improves fault tolerance.

- Provides faster access to frequently used data.

Example: A bank stores copies of customer account information at multiple branch servers.

C. Hybrid Distribution Model: The hybrid model combines both fragmentation and replication. Some data is partitioned across sites, while important data is replicated at multiple locations. **Advantages:**

- Provides both high performance and reliability.
- Balances storage efficiency and availability.
- Supports large-scale distributed systems.

Example: Regional sales data is fragmented by location, while product information is replicated across all branches.

Sharding

Sharding is a database scaling technique in which a large database is divided into smaller, independent parts called **shards**. Each shard contains a portion of the total data and is stored on a separate server. Together, all shards form a single logical database.

Sharding is commonly used in large-scale distributed databases to improve performance, scalability, and data management.

Sharding Process: The sharding process involves dividing data and distributing it across multiple database servers.

Steps in Sharding:

1. Select a **shard key** that determines how data will be distributed.
2. Divide the database into multiple shards based on the shard key.
3. Store each shard on a separate server or node.
4. Route incoming queries to the appropriate shard.
5. Combine results from different shards when necessary.

```
Customer Database
Shard 1 → Customers A-F
Shard 2 → Customers G-M
Shard 3 → Customers N-Z
```

Types of Sharding

i) Range-Based Sharding: In range-based sharding, data is divided according to a specified range of values in the shard key.

Advantages:

- Simple to implement.
- Efficient for range queries.

Disadvantages:

- Uneven data distribution may occur.
- Some shards may become overloaded.

```
Shard 1 → IDs 1-1000
Shard 2 → IDs 1001-2000
Shard 3 → IDs 2001-3000
```

ii) Hash-Based Sharding: In hash-based sharding, a hash function is applied to the shard key to determine the target shard.

Example: $\text{Shard Number} = \text{Hash}(\text{Customer_ID}) \% \text{Number of Shards}$

Advantages:

- Provides uniform data distribution.

- Prevents hotspot formation.

Disadvantages:

- Range queries become difficult.
- Resharding can be complex.

iii) Directory-Based Sharding: In directory-based sharding, a lookup table or directory maintains information about the location of each data item. The directory is consulted before accessing data.

Advantages:

- Flexible data placement.
- Easy to modify shard assignments.

Disadvantages:

- Requires maintenance of the directory.
- Directory failures can affect performance.

```
Customer A → Shard 1
Customer B → Shard 3
Customer C → Shard 2
```

Advantages

- **Scalability:** Supports growth by adding more servers.
- **Improved Performance:** Reduces workload on individual servers.
- **High Availability:** Failure of one shard does not affect the entire database.
- **Load Balancing:** Distributes data and traffic across multiple nodes.
- **Faster Queries:** Smaller datasets improve query execution speed.
- **Cost Efficiency:** Uses multiple low-cost servers instead of a single powerful server.

Challenges

- **Complex Management:** Shard administration can be difficult.
- **Data Rebalancing:** Redistribution is required when adding new shards.
- **Cross-Shard Queries:** Queries spanning multiple shards are more complex.
- **Uneven Distribution:** Some shards may store more data than others.
- **Backup Complexity:** Managing backups across shards can be challenging.
- **Increased Development Effort:** Applications must be designed to support sharding.

Replication

Replication is the process of creating and maintaining multiple copies of the same data across different database servers or nodes. The copies, known as replicas, are kept synchronized so that changes made to one copy are reflected in the others.

Replication is used in distributed database systems to improve data availability, reliability, fault tolerance, and performance. By storing data at multiple locations, replication ensures that the system can continue operating even if one server fails.

b) Need for Replication

Replication is important for several reasons:

- Improves data availability.
- Provides backup copies of data.
- Increases fault tolerance.
- Reduces the risk of data loss.
- Improves read performance.
- Supports disaster recovery.
- Enables load balancing.
- Ensures business continuity.

- Allows data access from multiple locations.
- Enhances system reliability.

c) Working of Replication

The replication process generally follows these steps:

1. Data is stored in a primary database.
2. Changes are made to the primary database.
3. The changes are recorded in transaction logs.
4. Replication mechanisms capture these changes.
5. Updates are transmitted to replica databases.
6. Replica databases apply the changes.
7. All copies remain synchronized.

Master-Slave Replication

Master-Slave Replication is a database replication technique in which one server, called the **Master**, manages all data modifications, while one or more **Slave** servers maintain copies of the master's data. The slaves receive updates from the master and keep their databases synchronized. This model is widely used to improve read performance, provide backup copies of data, and ensure higher availability in distributed database systems.

b) Architecture

The architecture consists of a central Master node and multiple Slave nodes.

Master Server

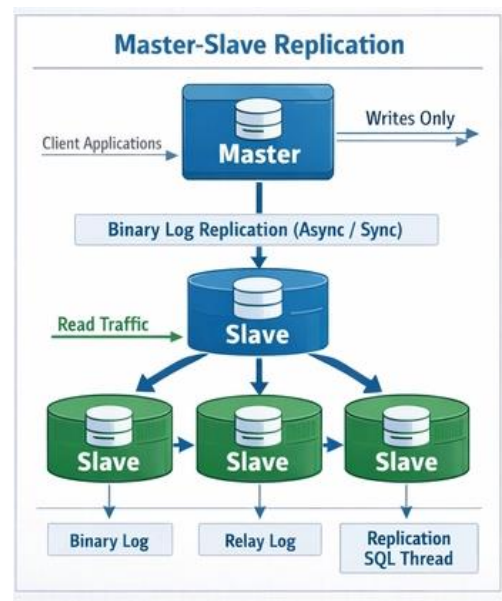
- Stores the primary copy of data.
- Handles INSERT, UPDATE, and DELETE operations.
- Records changes in transaction logs.
- Sends updates to Slave servers.

Slave Servers

- Maintain copies of the master's database.
- Receive updates from the Master.
- Primarily handle read requests.
- Provide backup and redundancy.

c) Working

- A client sends a write request to the Master server.
- The Master updates its database.
- The Master records the changes in a transaction log.
- Replication software transfers these changes to Slave servers.



- Each Slave applies the changes to its local database.
- Read operations can be performed from any Slave server.
- The process continues to keep all replicas synchronized.

d) Advantages

- Improves read performance by distributing queries among slaves.
- Reduces workload on the Master server.
- Provides backup copies of data.
- Enhances system availability.
- Supports disaster recovery.
- Easy to understand and implement.
- Allows load balancing of read operations.
- Improves response time for users.
- Suitable for large web applications.

- Helps maintain business continuity.

e) Limitations

- Master becomes a single point of failure.
- Write operations depend entirely on the Master.
- Replication delays may occur.
- Slaves may temporarily contain outdated data.
- Additional storage is required for replicas.
- Failover management can be complex.
- Not suitable for write-intensive applications.

- Network failures can affect synchronization.
- Recovery after Master failure may take time.
- Scalability of write operations is limited.

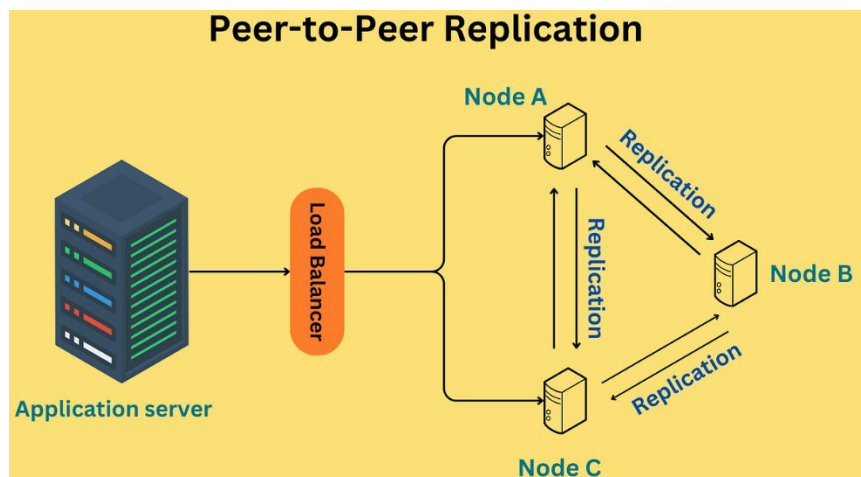
f) Applications of Master-Slave Replication

- E-commerce websites.
- Banking systems.
- Online reservation systems.
- Content management systems.
- Social networking platforms.
- Data warehousing environments.
- Large-scale web applications.
- Reporting and analytics systems.

Peer-to-Peer Replication

Peer-to-Peer Replication is a replication model in which all database nodes have equal status. Each node can perform both read and write operations and can share updates with other nodes.

Unlike Master-Slave Replication, there is no central controlling server. Every node acts as both a publisher and subscriber of data.



b) Architecture

The architecture consists of multiple interconnected database nodes.

- No dedicated Master server exists.
- Every node contains a copy of the database.
- Each node can process read and write requests.

- Data updates are exchanged among all participating nodes.
- Nodes communicate directly with one another.

c) Working

1. A user updates data on any node.
2. The node records the modification.

3. The update is transmitted to other nodes.
4. All nodes apply the received changes.
5. Data remains synchronized across the network.
6. If two nodes modify the same data simultaneously, conflict resolution mechanisms are applied.
7. Each node eventually reaches a consistent state.

d) Advantages

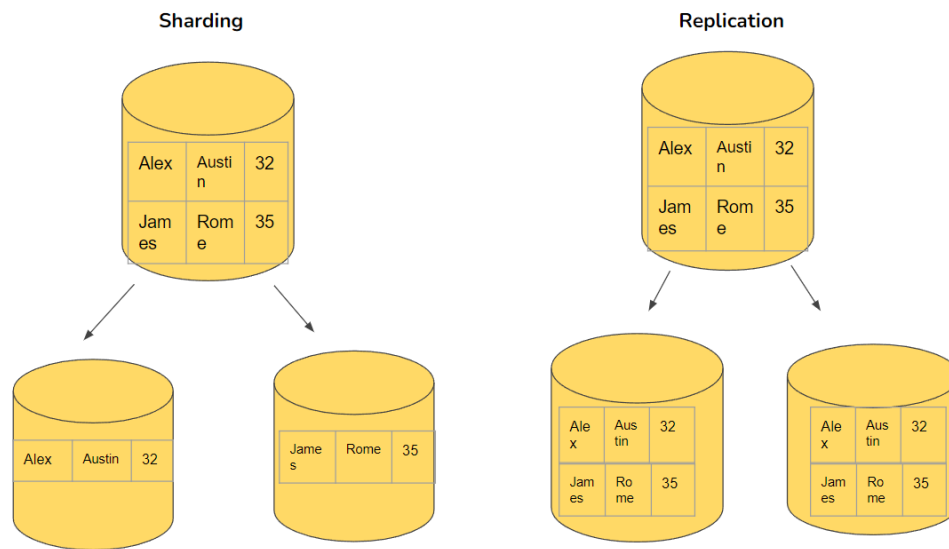
- Eliminates a single point of failure.
- Provides high system availability.
- Supports both read and write operations at every node.
- Improves fault tolerance.
- Enables geographical distribution of databases.
- Reduces dependency on a central server.
- Allows local access to data.
- Supports load sharing among nodes.

- Increases system reliability.
- Suitable for distributed enterprise environments.

e) Limitations

- Conflict detection and resolution can be difficult.
- Synchronization mechanisms are complex.
- Network traffic increases significantly.
- Maintaining data consistency is challenging.
- Administration becomes more complicated.
- Performance may decrease as the number of nodes grows.
- Troubleshooting replication issues can be difficult.
- Requires careful planning and monitoring.
- Higher implementation cost.
- Not ideal for applications requiring strict real-time consistency.

Basis	Replication	Sharding
Definition	Creates multiple copies of the same data.	Divides data into smaller parts called shards.
Purpose	Improves availability and fault tolerance.	Improves scalability and performance.
Data Storage	Same data exists on multiple servers.	Different data exists on different servers.
Data Distribution	Data is duplicated across nodes.	Data is partitioned across nodes.
Read Performance	Improved due to multiple replicas.	Improved as queries access smaller datasets.
Write Performance	Usually limited by synchronization.	Improved by distributing write operations.
Storage Requirement	High, due to duplicate copies.	Lower, as data is split among servers.
Fault Tolerance	High, because replicas act as backups.	Lower unless combined with replication.
Scalability	Mainly supports read scalability.	Supports both data and workload scalability.
Data Availability	High availability during failures.	Depends on shard availability.
Consistency Issues	Replication lag may occur.	Cross-shard consistency can be challenging.
Network Overhead	High due to synchronization traffic.	Lower compared to replication.
Implementation Complexity	Relatively simpler.	More complex due to partition management.
Example	Master-Slave Replication.	Range-Based or Hash-Based Sharding.

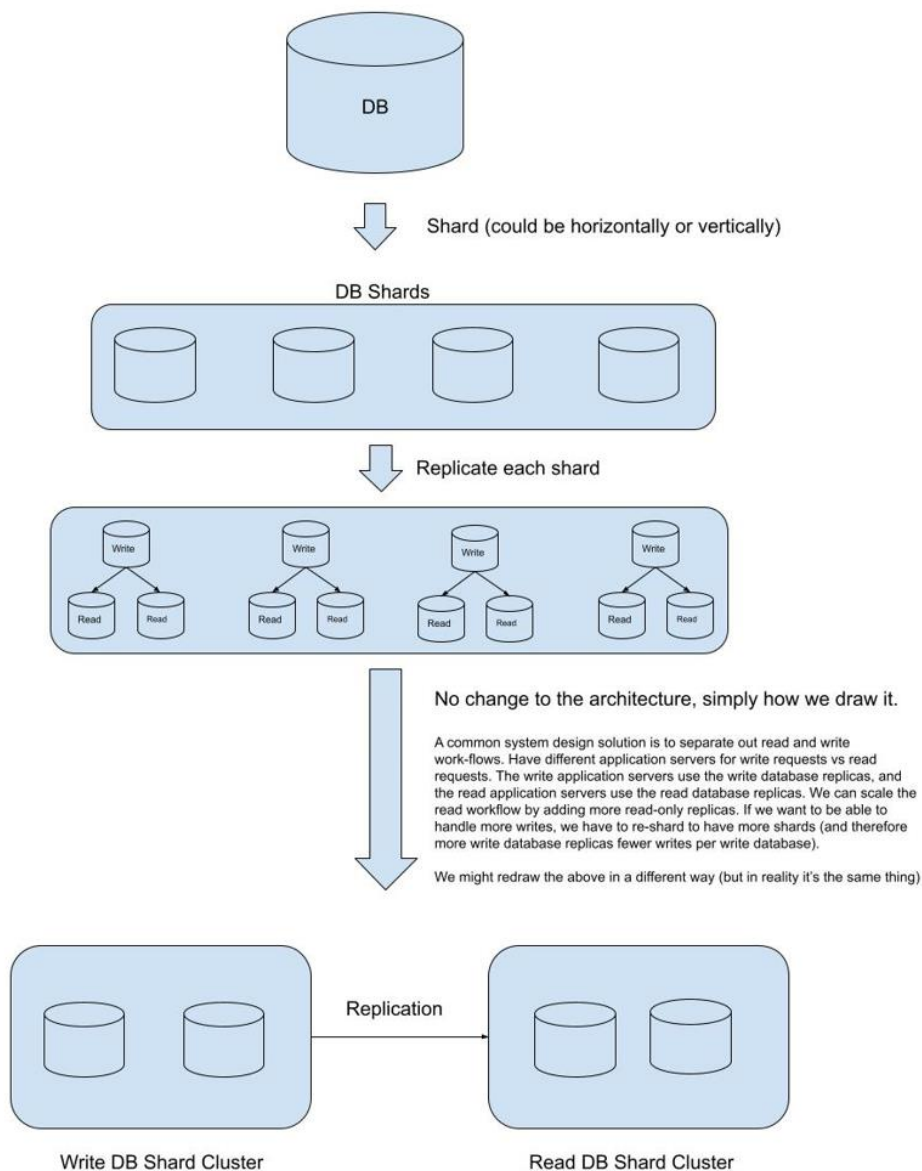


Combining Sharding and Replication

Sharding and Replication are often used together in distributed database systems to achieve both scalability and high availability. **Sharding** divides a large database into smaller partitions called **shards**, while **Replication** creates multiple copies of each shard on different servers.

In this approach:

- Data is first divided into multiple shards.
- Each shard is stored on a separate server.
- Copies of each shard are maintained on replica servers.
- One replica may act as the primary node, while others act as secondary nodes.
- Queries are distributed across shards and replicas.



a) Working

1. Data is divided into multiple shards.
2. Each shard is assigned to a server.
3. Copies of every shard are created on replica servers.
4. Write operations are performed on the primary replica.
5. Changes are synchronized to secondary replicas.
6. Read requests may be served by replicas.
7. If one server fails, a replica can take over.

This combination provides both horizontal scaling and fault tolerance.

b) Benefits of the combination

- **Scalability:** Supports growth by adding new shards.
- **High Availability:** Replicas keep data accessible during failures.
- **Better Performance:** Queries run faster on smaller datasets.
- **Load Balancing:** Workload is distributed across servers.
- **Fault Tolerance:** System continues operating despite node failures.

- **Data Redundancy:** Multiple copies reduce data loss risk.
- **Faster Reads:** Replicas handle read requests efficiently.
- **Geographic Distribution:** Data can be stored closer to users.
- **Improved Reliability:** Multiple nodes increase system dependability.
- **Supports Big Data:** Handles massive volumes of data effectively.

- **Higher Storage Cost:** Replicas require additional storage space.
- **Network Overhead:** Synchronization generates extra network traffic.
- **Failover Complexity:** Recovery procedures can be difficult to manage.
- **Monitoring Requirements:** Multiple servers require continuous monitoring.
- **Maintenance Effort:** System administration becomes more demanding.

c) Challenges

- **Complex Management:** Combining sharding and replication increases complexity.
- **Cross-Shard Queries:** Queries across shards are difficult to execute.
- **Data Rebalancing:** Shards must be redistributed when scaling.
- **Replication Lag:** Replicas may not reflect updates immediately.
- **Consistency Issues:** Maintaining identical data across replicas is challenging.

d) Applications of Sharding with Replication

- Large-scale social media platforms.
- E-commerce websites.
- Cloud databases.
- Banking and financial systems.
- Online gaming platforms.
- Big data analytics systems.
- Streaming services.
- Global web applications.

Consistency

Consistency is the property of a distributed database that ensures all users see the same and most recent data across all nodes. After a data update, every replica should eventually reflect the same value. Consistency ensures that data remains accurate, valid, and synchronized throughout the system.

b) Importance of Consistency

- Ensures data accuracy across all nodes.
- Prevents conflicting data values.
- Maintains data integrity.
- Improves reliability of transactions.
- Provides a uniform view of data.
- Reduces chances of incorrect decisions.
- Supports application correctness.
- Essential for financial and banking systems.
- Helps maintain user trust.
- Ensures proper synchronization of replicas.

c) Consistency Models

Consistency models define how and when updates become visible to users in a distributed database.

i) Strong Consistency

Strong Consistency guarantees that every read operation returns the most recent write. Once data is updated, all users immediately see the updated value regardless of the node they access.

Characteristics

- Always returns the latest data.
- Ensures a single consistent view.
- No stale data is observed.
- All nodes remain synchronized.

Advantages

- High data accuracy.
- Simplifies application development.
- Suitable for critical applications.
- Prevents data conflicts.

Limitations

- Higher response time.
- Increased communication overhead.
- Reduced system availability during failures.

Applications

- Banking systems.
- Online payment systems.
- Stock trading platforms.
- Reservation systems.

ii) Weak Consistency

Weak Consistency does not guarantee that all users immediately see the latest update. Data updates may take time to propagate across nodes.

Characteristics

- Updates are not instantly visible.
- Different nodes may contain different values temporarily.
- Consistency is not guaranteed after every write.

Advantages

- Faster response time.
- Lower communication overhead.
- Better system performance.

Limitations

- Users may read outdated data.
- Temporary inconsistencies may occur.
- Not suitable for critical transactions.

Applications

- Caching systems.
- Multimedia streaming.
- Social media feeds.
- Online gaming systems.

iii) Eventual Consistency

Eventual Consistency is a consistency model in which all replicas eventually become consistent if no new updates occur. Temporary inconsistencies are allowed, but the system guarantees convergence over time.

Characteristics

- Updates propagate gradually.
- Temporary inconsistencies are allowed.
- All replicas eventually reach the same state.
- Common in large-scale distributed systems.

Advantages

- High availability.
- Better scalability.
- Improved performance.

- Supports geographically distributed databases.

Limitations

- Users may temporarily see stale data.
- Conflict resolution may be required.
- Consistency is not immediate.

Applications

- NoSQL databases.
- Cloud storage systems.
- E-commerce platforms.
- Distributed web applications.
- Social networking platforms.

Comparison of Consistency Models

Feature	Strong Consistency	Weak Consistency	Eventual Consistency
Latest Data Guarantee	Always	No	Eventually
Data Accuracy	Very High	Low	Moderate
Response Time	Slower	Faster	Fast
Availability	Lower	Higher	High
Scalability	Limited	High	Very High
Temporary Inconsistency	Not Allowed	Allowed	Allowed
Suitable For	Banking, Payments	Caching, Gaming	NoSQL, Cloud Systems

Relaxing Consistency

Relaxing Consistency is an approach in distributed database systems where strict consistency requirements are reduced to improve system performance, availability, and scalability. Instead of ensuring that all nodes immediately reflect every update, temporary inconsistencies are allowed, with the expectation that data will become consistent later.

This approach is widely used in large-scale distributed systems where immediate consistency is less important than system responsiveness and availability.

b) Need for Relaxing Consistency

- Improves system availability.
- Reduces response time.
- Enhances scalability.
- Supports geographically distributed systems.
- Minimizes communication overhead.
- Handles network delays efficiently.
- Improves user experience.
- Supports large numbers of users.
- Reduces synchronization costs.
- Enables continuous system operation during failures.
- Improves system performance.
- Supports horizontal scalability.
- Reduces network overhead.
- Allows faster data access.
- Handles large workloads efficiently.
- Supports distributed environments.
- Improves fault tolerance.

c) How Relaxing Consistency Works

1. Data is updated on one node.
2. The update is accepted immediately.
3. Other nodes receive the update later.
4. Temporary differences may exist among replicas.
5. Synchronization occurs in the background.
6. All replicas eventually become consistent.

d) Advantages

- Provides high availability.

e) Challenges

- Users may read stale data.
- Temporary inconsistencies can occur.
- Conflict resolution may be required.
- Data synchronization becomes complex.
- Application logic may be more difficult.
- Consistency guarantees are reduced.
- Debugging can be challenging.
- Not suitable for critical transactions.

f) Applications

- Social networking platforms.
- Cloud storage systems.
- NoSQL databases.
- E-commerce websites.
- Content delivery networks (CDNs).
- Online gaming platforms.

- Distributed web applications.
- Large-scale internet services.

Example

Suppose a user updates their profile picture on a social media platform.

- The update is saved on one server immediately.

- Some users may still see the old picture for a short time.
- After synchronization, all servers display the new picture.
- The system achieves consistency eventually rather than instantly.

Version Stamps

A Version Stamp is a unique identifier or version number associated with a data item to track changes made to it over time. Whenever the data is modified, its version stamp is updated.

Version stamps help distributed databases determine whether data has changed and identify the most recent version of a record.

b) Purpose

- Tracks changes made to data.
- Identifies the latest version of a record.
- Detects data conflicts.
- Supports synchronization between replicas.
- Maintains data consistency.
- Prevents overwriting of recent updates.
- Assists in concurrency control.
- Improves data integrity.

c) Working

1. A record is assigned an initial version stamp.
2. When the record is updated, the version stamp changes.
3. Replicas compare version stamps during synchronization.
4. The record with the newer version stamp is identified.
5. Conflicts can be detected using version differences.
6. Replicas update themselves with the latest version.
7. Consistency is maintained across the system.

Example

Record	Data	Version Stamp
Customer 101	Kolkata	V1
Customer 101	Mumbai	V2

Here, **V2** indicates that the record has been updated after **V1** and is therefore the latest version.

d) Advantages

- Easily identifies updated records.
- Simplifies synchronization.
- Helps detect update conflicts.
- Supports distributed databases.
- Improves consistency management.
- Prevents data overwrite issues.
- Enhances concurrency control.
- Reduces synchronization errors.
- Maintains data integrity.
- Improves reliability of replicated systems.